Solutions: Day 35 Practice Problems

Complete, production-tested solutions and architectural walkthroughs for Day 35 Git & GitHub challenges.

Solution 1: Production Hotfix & Branch Isolation Pipeline

1. Inspect Active Branch & Status

```
git branch
git status
```

- **Explanation**: `git branch` displays the list of branches with `*` indicating your active branch (`feature-fraud-v2`). `git status` reports untracked, modified, and staged files.

2. Safe Context Switching

- **The Risk**: If you run `git checkout main` while having uncommitted modifications, Git may either:
    1. Abort the checkout with error: `error: Your local changes to the following files would be overwritten by checkout` .
    2. Or carry the uncommitted dirty working tree changes into `main` , contaminating production code with unfinished experimental code.
- **Save Work via Commit**:

    ```
    git add .
    git commit -m "WIP: Initial feature engineering for fraud detection v2"
    ```

    (Alternative: `git stash` to temporarily shelf changes without creating a formal commit).

3. Hotfix Branch Creation & Fix

```
# Step 1: Switch to main and sync with remote
git checkout main
git pull origin main

# Step 2: Create and switch to hotfix branch
git checkout -b hotfix-auth-patch
```

```
# Step 3: Implement fix in auth.py, stage, and commit
git add auth.py
git commit -m "Patch CVE auth token bypass"
```

4. Integration, Deployment & Cleanup

```
# Step 1: Switch to main and merge the patch
git checkout main
git merge hotfix-auth-patch

# Step 2: Push production fix to GitHub
git push origin main

# Step 3: Delete local hotfix branch (safe deletion)
git branch -d hotfix-auth-patch

# Step 4: Return to development branch
git checkout feature-fraud-v2
```

Solution 2: Multi-Developer Merge Conflict Resolution

1. Conflict Marker Anatomy

Inside `database.py`, Git inserts conflict delimiters:

```
<<<<<<< HEAD
DB_PORT = 5433
DB_POOL_SIZE = 50
=======
DB_PORT = 5432
DB_POOL_SIZE = 20
>>>>>>> origin/main
```

- `<<<<<<< HEAD`: Start of your local changes currently checked out in your working directory.
- `=======`: The delimiter dividing your version from the incoming version.
- `>>>>>>> origin/main`: End of the incoming commit received from GitHub.

2. Manual Resolution

According to team consensus (Port `5432`, Pool `50`), open `database.py`, delete all conflict markers (`<<<<<<<`, `=======`, `>>>>>>>`), and combine the desired values:

```
DB_PORT = 5432
DB_POOL_SIZE = 50
```

3. Finalizing the Merge

```
# Verify modified status
git status

# Stage the resolved file
git add database.py

# Finalize the merge commit
git commit -m "Resolve merge conflict in database.py: keep port 5432 and pool 50"

# Push resolution to GitHub
git push origin main
```

Solution 3: Disaster Recovery & Accidental Secret Commit Mitigation

1. Immediate Critical Action: Revoke & Invalidate Credentials

Rule 1 of Security: The moment a credential touches a public Git repository, consider it compromised. Automated scraping bots continuously crawl GitHub's public event stream within milliseconds.

- **First Action**: Immediately log into the OpenAI Developer Dashboard and AWS IAM Console and **revoke / delete the compromised API keys**. Generate new replacement keys.
- Rewriting Git history alone does not guarantee security because web crawlers or caching mirrors may have already indexed the commit.

2. Untracking `.env` Without Deleting Physical Local File

```
git rm --cached .env
```

- **Explanation**:
  - `git rm .env`: Deletes the file from both Git tracking and your physical hard drive.
  - `git rm --cached .env`: Removes the file from Git's staging area (index) while keeping the physical file intact on disk.

3. Adding `.env` to `.gitignore`

Append `.env` patterns to `.gitignore`:

```
# Ignore environment secrets
.env
.env.*
!.env.example
```

(Tip: Always commit a sanitized `.env.example` file showing variable names with dummy placeholder values so team members know what configuration keys are required).

4. History Remediation: `git revert` vs `git reset`

- **Using `git revert`**:

  ```
  git revert <bad_commit_hash>
  git push origin main
  ```

  - **Pros**: Safe, preserves linear history, does not disrupt teammates.
  - **Cons**: The bad commit containing the raw secret still remains visible in past commit history. (Hence credential revocation is mandatory).
- **Using `git reset --hard` & Force Push**:

  ```
  git reset --hard HEAD~1
  git push origin main --force
  ```

  - **Dangers**: A force push rewrites remote branch history, which overwrites remote commits and desynchronizes clones across all team members' computers.
  - **Conclusion**: Regardless of whether you reset or revert, credentials must always be revoked immediately.